

시나리오 데이터 구조 설계 방법론 요약

인크리멘탈 스토리 게임 「프로젝트_출근」 담당 구간

담당	시나리오 데이터 구조 · 이벤트 조건 설계
기간	2026.08 ~ 2026.09
규모	기획 3 / 개발 1
작성	서윤준 · koioas1@gmail.com

CONTENTS

- 문제 정의 흐름 제어가 없는 구조 위에서 무엇을 만들어야 했는가
- 데이터 구조 — 3 테이블 역할 분리 책임 분리 · 고정 슬롯 제약 · 식별자 명명 규칙
- 판정 시점과 조건식 문법 하루 루프의 단계 구성과 최소 어휘 조건식
- 분기 해법 — 조건 테이블 이어받기 패턴 복제하지 않고 분기를 만드는 방법
- 반복 구조를 건디게 하는 랜덤 대사 적재 적재 · 소모 · 정리와 수치 없는 호감도 전달
- 판정 구간 개편과 데이터 마이그레이션 스크립트 일괄 변환과 정합성 자동 검사
- 산출물과 회고 담당 구간 산출물과 배운 점

문서 안내. 이 문서는 사내 기획 문서의 발췌본이 아니라, 포트폴리오 공개를 위해 새로 작성한 방법론 요약본입니다. 진행 중인 사업과 관련된 내용을 보호하기 위해 다음 항목은 의도적으로 제외했습니다 — 회사명 및 고객사 정보, 작품의 스토리·설정·엔딩 내용, 등장인물의 실제 이름, 실제 대사 원문, 사내 문서의 서술과 표 형식. 본문에 등장하는 모든 식별자·수식·대사·수치 예시는 구조를 설명하기 위해 **가상으로 새로 작성한 것**이며 실제 데이터가 아닙니다. 설계 규칙과 방법론, 그 리고 작업 규모를 나타내는 개략 수치만 서술합니다.

01 문제 정의

담당 업무는 대사를 쓰는 일이 아니라, 어떤 대사가 언제 어떤 조건으로 출력될지를 규정하는 데이터 구조를 세우는 일이었습니다.

주어진 제약

엔진은 세 개의 데이터 테이블만 읽습니다. 시퀀스는 순서대로 실행되는 액션의 목록이고, 조건 테이블은 특정 시점에 어떤 시퀀스를 실행할지 판정하며, 대사 테이블은 화면에 출력할 텍스트를 갖습니다. 여기서 설계상 가장 큰 제약은 **시퀀스가 흐름 제어를 할 수 없다는 점**이었습니다. 시퀀스 내부에는 조건 분기도, 반복도, 조기 종료도 없습니다. 위에서 아래로 전부 실행됩니다.

목표

- 6주 분량의 하루 루프를 반복 구조로 구성하되, 매일이 같아 보이지 않게 할 것
- 플레이어의 과거 선택이 이후 대화에 반영되게 할 것
- NPC별 호감도를 **숫자 UI 없이** 대사의 형태 변화만으로 전달할 것
- 데이터가 수만 행으로 불어나도 한 사람이 유지보수할 수 있게 할 것

네 번째 항목이 실질적으로 모든 설계 판단의 기준이 됐습니다. 분기를 늘리는 가장 쉬운 방법은 경로마다 데이터를 복제하는 것이지만, 그 방식은 수정 한 번에 복제본 전부를 손대야 하므로 규모가 커지는 순간 붕괴합니다. 이 문서의 나머지는 **복제하지 않고 분기를 만드는 방법**에 대한 기록입니다.

02 데이터 구조 — 3 테이블 역할 분리

각 테이블에 단 하나의 책임만 부여하고, 테이블 간 참조는 ID로만 연결했습니다.

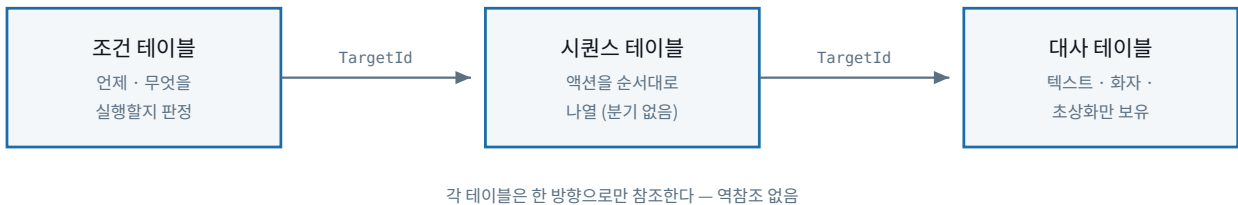


그림 1. 3 테이블의 역할 분리와 참조 방향

테이블별 책임

테이블	책임	주요 필드
조건 (Condition)	판정 시점마다 실행할 시퀀스를 결정	Uid · TargetId · Priority · Day · Timing · Condition
시퀀스 (Sequence)	액션을 순서대로 실행	Uid · Order · ActionType · TargetId · StringValue00~04 · FloatValue00~04
대사 (Dialog)	출력 텍스트와 화자 표현만 보유	Uid · PortraitPosition · PortraitPath · CharacterName · Dialog

고정 파라미터 슬롯이라는 제약

시퀀스의 액션은 종류가 무엇이든 문자열 5칸·실수 5칸의 고정 슬롯만 사용합니다. 액션마다 필드를 새로 만들 수 없으므로, **액션별로 슬롯의 의미를 규정한 대응표**가 사실상 데이터 규격의 핵심이 됩니다. 이 표가 흔들리면 같은 컬럼이 액션마다 다른 뜻이 되어 데이터 전체를 사람이 읽을 수 없게 됩니다.

액션 (가상 예시)	String00	String01	Float00	Float01
Wait	—	—	대기 시간	—
Dialog	—	—	—	—
ChangeBackground	스프라이트 경로	색상 코드	전환 시간	—
Overlay	스프라이트 경로	색상 코드	대기 시간	페이드 인
MoveNpc	대상 캐릭터	대기 여부	X 좌표	Y 좌표

표의 액션 구성과 슬롯 배치는 설명을 위해 재구성한 예시입니다.

식별자 명명 규칙

모든 행은 정렬만으로 소속과 시점을 알 수 있어야 합니다. 접두어로 테이블 종류를, 이어지는 토큰으로 일자·장소·일련번호를 표기하는 규칙을 정하고 예외를 두지 않았습니다.

[테이블접두어]_D[일자 2자리]_[장소]_[일련번호 3자리]_[변형]

예시(가상) seq_D01_Office_014 시퀀스 · 1일자 · 사무실 · 14번
 con_D01_112_Office 조건 · 1일자 · 우선순위 112
 dlg_D01_Office_014_Good 대사 · 1일자 · 사무실 · 14번 · 호감 상위 변형

규칙 · 변형 접미어는 사전에 정의된 어휘만 사용 (_Good / _Bad / _VeryGood 등)
 · 일련번호는 재사용하지 않는다 - 삭제된 번호는 영구 결번
 · 접두어 없는 식별자는 임포트 단계에서 오류로 처리

결번 원칙의 이유. 조건식이 특정 대사의 재생 여부를 참조하기 때문에, 번호를 재사용하면 삭제된 대사를 가리키던 조건식이 **조용히 다른 대사를 가리키게** 됩니다. 오류가 나지 않고 동작만 달라지므로 발견이 늦고, 그래서 규칙 단계에서 원천 차단했습니다.

03 판정 시점과 조건식 문법

하루를 여러 개의 고정 단계로 쪼개고, 각 단계 진입 시점마다 조건 테이블을 조회하는 구조를 잡았습니다.



각 지점 진입 시 조건 테이블 조회 → 조건을 만족한 행을 우선순위 순서대로 연속 실행

그림 2. 하루 루프의 판정 시점 (단계 구성은 설명용으로 축약)

조건식 문법

조건식은 개발자가 함수 몇 개만 구현하면 되도록 최소 어휘로 정의했습니다. 기획자가 스프레드시트에서 직접 쓰는 식이므로, 문법이 늘어날수록 오타로 인한 사고가 늘어난다고 판단했습니다.

판정 함수	GetReputation("NPC") IsPlayedSequence("ID") IsPlayedDialog("ID")	해당 NPC의 평판을 정수로 반환 해당 시퀀스의 실행 여부 해당 대사의 출력 여부
연산자	and or not () >= <= > < ==	
가상 예시	GetReputation("NPC_A") >= 55 and GetReputation("NPC_A") <= 74 not IsPlayedDialog("dlg_D01_Office_014") and GetReputation("NPC_A") >= 55 (IsPlayedSequence("seq_D02_Office_101") or IsPlayedSequence("seq_D02_Office_102")) and GetReputation("NPC_B") >= 50	

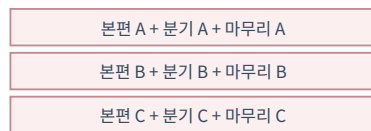
설계 원칙 세 가지

1. **상태를 따로 저장하지 않는다.** 과거 선택은 별도 플래그 변수가 아니라 "그 시퀀스가 실행됐는가"로 판정합니다. 플래그를 두면 세우는 것을 잊은 지점에서 조용히 어긋납니다.
2. **구간은 반드시 상·하한을 함께 쓴다.** 상한을 생각하면 상위 구간이 하위 구간 조건에도 걸려 두 대사가 함께 출력됩니다. 이 형태의 사고가 초기에 반복돼 규칙으로 고정했습니다.
3. **같은 시점의 조건은 우선순위로 전순서를 갖는다.** 동순위를 허용하지 않아 실행 순서가 데이터 정렬 방식에 의존하지 않도록 했습니다.

04 분기 해법 — 조건 테이블 이어받기 패턴

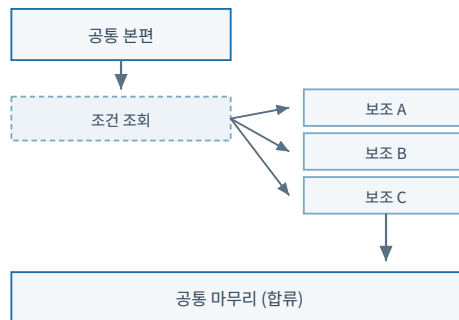
문제	시퀀스는 흐름 제어를 할 수 없으므로, 전날의 선택에 따라 다음 날 대화를 갈라지게 하려면 해당 구간 전체를 경로 수만큼 복제하는 방법밖에 없었습니다. 경로가 3개면 본편이 3벌이 되고, 대사 한 줄을 고칠 때마다 3곳을 고쳐야 합니다.
해결	한 판정 시점에 조건행이 여러 개 연속으로 실행된다 는 성질을 이용했습니다. 본편 시퀀스를 분기 지점에서 끊고, 그 자리를 조건 테이블이 이어받아 경로별 보조 시퀀스를 재생한 뒤, 다시 공통 마무리 시퀀스로 합류시키는 구조입니다. 경로 판정은 과거 시퀀스의 실행 여부를 묻는 조건식으로 처리했습니다.
결과	본편을 복제하지 않고 분기를 확보했습니다. 경로가 하나 늘어도 추가되는 것은 보조 시퀀스 한 벌과 조건행 한 줄이며, 공통 구간은 계속 한 벌로 유지됩니다. 이 패턴은 이후 과거 선택을 참조해야 하는 모든 구간의 표준 방식으로 정착했습니다.

복제 방식 — 경로마다 전체를 복사



수정 1회 = 3곳 수정

이어받기 방식 — 공통 구간은 한 벌



수정 1회 = 1곳 수정 · 경로 추가 = 보조 1벌 + 조건 1행

그림 3. 복제 방식과 이어받기 방식의 비교

05 반복 구조를 견디게 하는 랜덤 대사 적재

6주 동안 같은 하루 루프가 반복되므로, 같은 시점의 대사가 매년 같으면 3일차에 이미 지겨워집니다.

적재 · 소모 · 정리의 3단 구조

NPC별 대사 후보를 **가중치와 함께 미리 적재**해 두고, 게임이 그 중 하나를 뽑아 쓰고, 구간이 끝나면 전량 정리하는 방식으로 설계했습니다. 적재 단계에서 조건식을 함께 넣을 수 있으므로, "지금 이 NPC와의 관계에서 나올 수 있는 말"만 후보에 들어갑니다.

적재 AllocRandDialog 대상NPC · 대사ID · 조건식 · 후속시퀀스 · 가중치
소모 게임이 후보군에서 가중치에 따라 하나를 선택해 출력
정리 ClearRandDialog 대상NPC - 구간 전환 시 후보군 전량 초기화

가상 예시 (오전 업무 진입 시 NPC_A 후보 적재)

```
AllocRandDialog NPC_A dlg_..._001 GetReputation("NPC_A") < 20 - 1.0  
AllocRandDialog NPC_A dlg_..._014 GetReputation("NPC_A") >= 55 - 1.0  
AllocRandDialog NPC_A dlg_..._021 IsPlayedSequence("seq_...") - 0.5
```

정리 단계를 반드시 짝으로 두는 이유. 후보군이 초기화되지 않으면 오전 구간의 대사가 오후에 튀어나옵니다. 발생 빈도가 낮고 재현이 어려워 사람이 찾기 가장 힘든 종류의 버그이므로, 적재와 정리를 **짝으로 검사**하는 항목을 후술할 자동 검사 도구에 넣었습니다.

호감도를 숫자 없이 전달하기

평판 수치는 UI에 노출하지 않기로 했습니다. 대신 구간별로 다른 후보군이 적재되도록 해서, 플레이어가 **대사의 형태 변화**로 관계 변화를 감지하게 만들었습니다. NPC마다 변화의 축을 다르게 잡아 개성을 부여했습니다.

대상	변화의 축	플레이어가 체감하는 신호
NPC_A	호칭	부르는 말이 바뀐다
NPC_B	답변 길이	단답이 설명으로 늘어난다
NPC_C	요청 빈도	사적인 부탁이 늘어난다

실제 인물명과 대사는 제외했으며, 축의 종류만 표기했습니다.

평판이 움직이는 조건

대답 한 줄로는 수치가 오르지 않도록 설계했습니다. **상대가 그 선택을 관측했는가, 그 선택에 비용이 있었는가** — 두 조건을 모두 만족할 때만 증감이 발생합니다. 이 규칙이 없으면 플레이어는 모든 선택지에서 우호적인 답만 고르는 지배 전략을 찾아내고, 그 순간 관계 시스템 전체가 무의미해집니다.

06 판정 구간 개편과 데이터 마이그레이션

문제	평판 판정 구간을 4단계에서 5단계로 개편하기로 결정되면서, 데이터 전반에 상수로 박혀 있던 경계값 조건식이 한꺼번에 무효가 됐습니다. 대상은 1,400여 개 셀 . 스프레드시트에서 손으로 고치면 누락과 오타로 정합성이 깨질 것이 분명했습니다.
해결	수작업을 금지하고 Python(openpyxl) 마이그레이션 스크립트 로 일괄 치환했습니다. 조건식을 문자열로 치환하지 않고 구간 경계값을 파싱해 새 구간에 재배치하는 방식으로 작성해, 상·하한이 함께 이동하도록 했습니다. 변환 전후 비교표를 별도로 출력해 표본 검수했습니다.
결과	전량 변환 후 정합성 검사 전 항목을 0건으로 통과했습니다. 이후 모든 데이터 수정은 스크립트로 바꾸고 검사 도구로 확인한다 를 원칙으로 굳혔고, 개편 때마다 반복되던 수동 검수 공수를 제거했습니다.

정합성 자동 검사 항목

임포트 전에 반드시 실행하는 검사 도구를 별도로 만들었습니다. 아래는 검사 항목의 성격을 분류한 것으로, 각 항목은 위반 행의 식별자와 위치를 함께 출력합니다.

분류	검사 내용
식별자 무결성	식별자 중복, 명명 규칙 위반, 결번 재사용 여부
참조 무결성	존재하지 않는 시퀀스·대사를 가리키는 참조, 어디에서도 참조되지 않는 고아 행
조건식 정합	문법 오류, 미정의 함수·NPC 키, 구간 상·하한 누락, 구간 간 중첩
짜 맞추	적재와 정리, 연출의 표시와 해제가 짝을 이루는지
표기 일관성	화자 이름과 초상화 경로의 불일치, 좌우 배치 규칙 위반
실행 재현	조건 테이블을 시점별로 재현 실행해 도달 불가능한 행 검출

마지막 **실행 재현** 항목이 가장 효과가 컸습니다. 문법도 참조도 정상인데 상위 우선순위 조건에 항상 가려져 **영원히 실행되지 않는 행**은 사람 눈으로는 절대 찾을 수 없습니다. 조건 테이블을 실제 판정 순서대로 재현 실행해 보는 검사를 넣고 나서야 이 부류의 문제를 잡을 수 있었습니다.

07 산출물과 회고

담당 구간 산출물

산출물	내용
데이터 규칙서	액션별 파라미터 슬롯 대응표, 식별자 명명 규칙, 조건식 문법, 수치 증감 규정을 단일 정본으로 정립
시퀀스 데이터	2만여 행 — 6주 분량 하루 루프의 액션 목록
조건 데이터	2천여 행 — 판정 시점별 실행 조건과 우선순위
대사 데이터	1만여 행 — 화자·초상화·텍스트
마이그레이션 스크립트	구간 개편에 따른 조건식 일괄 변환 및 변환 전후 비교표 출력
정합성 검사 도구	6개 분류의 자동 검사, 위반 행 식별자·위치 리포트
이벤트 분기 설계	평판 구간과 진척 지표를 조합한 종결 분기의 진입 조건 설계

행 수는 작업 규모를 나타내기 위한 개략치입니다.

배운 점

수치를 넣은 뒤에는 반드시 모든 종단 경로를 따라가며 누적값을 계산해야 한다. 설계상 맞아 보이는 값도 실제 경로를 전수로 돌려 보면 특정 분기에서만 의도와 다르게 수렴하는 경우가 있었습니다. 이후로는 계산을 거치지 않은 수치는 확정하지 않는 습관이 생겼습니다.

규칙은 사람의 주의력이 아니라 도구가 지키게 해야 한다. 결번 원치도, 구간 상·하한 규칙도, 적재·정리 짝 맞추도 처음에는 문서에만 적혀 있었고 그때는 예외 없이 어겨졌습니다. 검사 도구에 항목으로 들어간 뒤에야 규칙으로 기능했습니다.

데이터 규모가 커질수록 기획자의 일은 콘텐츠 제작에서 구조 유지로 옮겨간다. 후반부 작업 시간의 상당 부분은 대사를 쓰는 것이 아니라 이미 쌓인 데이터가 계속 정합한 상태를 유지하도록 도구를 만들고 돌리는 데 들어갔습니다. 다음 프로젝트에서는 이 도구들을 **작업 시작 시점에** 먼저 만들 계획입니다.